

Lab 3 - ELA408

Kim Svedberg



I. PATH PLANNING

Dijkstra and A* are both graph based search algorithms that treat the environment as a discrete grid or graph and explore nodes until the goal is found. Dijkstra's algorithm guarantees a shortest path by expanding nodes in order of accumulated cost from the start [1]. A* is similar to Dijkstra but also adds a heuristic estimate cost of the remaining distance to the goal, allowing it to focus the search and reduce the number of nodes expanded [2]. For this project the Euclidean distance was chosen as the heuristic [3].

A. Advantages and Disadvantages

Dijkstra's algorithm: The main advantage of Dijkstra is its simplicity and guarantee of shortest paths on any non-negative weighted graph [4]. It requires no prior knowledge of the environment, making it a reliable baseline to many other path planners. It can also handle varying and asymmetric edge weights.

The main disadvantage is that nodes are expanded uniformly in all directions with no goal directed guidance, which is computationally expensive in large graphs. Additionally, it can't adapt to changes in the environment without a full re-computation from scratch, making it poorly suited for dynamic or unknown settings.

A* algorithm: A* is significantly more efficient than Dijkstra when a good heuristic is available, while still guaranteeing shortest paths provided the heuristic is admissible [5]. A more informed heuristic reduces search effort without sacrificing correctness, and A* is particularly effective in static, well-defined environments such as grid maps.

The disadvantages are that performance is similar to that of Dijkstra when the heuristic is weak or uninformative ($h \approx 0$). Like Dijkstra, it is not suited for dynamic environments, since any change in edge costs requires a full replan. Memory usage can also be significant in large search spaces, as all open and closed nodes must be retained. Finally, designing an admissible heuristic can be hard in complex or high dimensional spaces.

B. Results

The resulting paths for Dijkstra and A* algorithm for a known environment can be seen in Figure 1 and 2, respectively. Both algorithms successfully reached the goal from the starting position. The number of pushes and pops for both algorithms for a known environment can be seen in Table I, with Dijkstra needing about 5 times the pushes and pops compared to A*.

Ksg22001@student.mdu.se

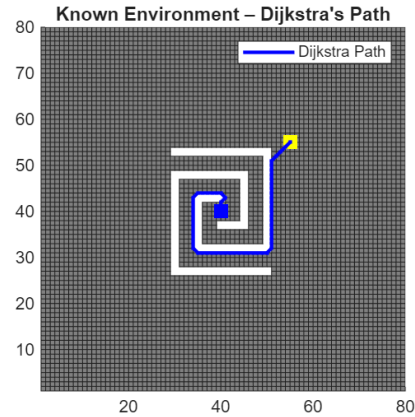


Fig. 1. Dijkstra path planning for a known environment.

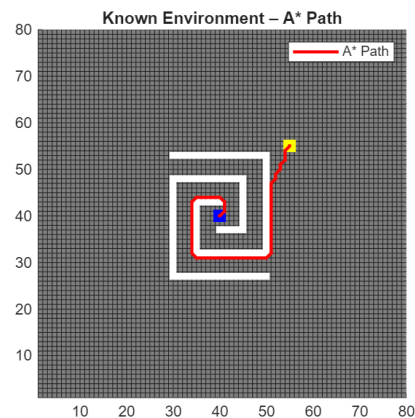


Fig. 2. A* path planning for a known environment.

The resulting paths for Dijkstra and A* algorithm for an unknown environment, where obstacles are not initially known but are discovered as the robot travels, can be seen in Figure 3 and 4, respectively. Both algorithms successfully reached the goal from the starting position. The number of pushes and pops for both algorithms for an unknown environment can be seen in Table II, with Dijkstra needing almost 7 times the pushes and 12 times the pops compared to A*.

C. Discussion

In the known environment, A* is much more efficient compared to Dijkstra because of the heuristic guidance. Rather than expanding nodes uniformly in all directions, A* focuses expansion toward the goal, deprioritizing nodes that lie far from the goal directed path. Dijkstra, not doing this, explores a large portion of the free space before terminating.

The gap widens considerably in the unknown environment. When a new obstacle is encountered and a step is taken, both

TABLE I
NUMBER OF PUSHES AND POPS - KNOWN ENVIRONMENT.

Algorithm	Pushes	Popps
Dijkstra	3392	3097
A*	687	626

algorithms must replan. Because Dijkstra maintains a broad search across the entire explored space, re-expansion propagates through many previously visited nodes. A* maintains a much narrower search window directed toward the goal, so far fewer nodes are affected upon replanning. This explains why the pop ratio scales from about 5 times in the known case to 12 times in the unknown case, the heuristic provides not only an increase in efficiency but also robustness to replanning. These results shows that A* should be preferred for robot path planning whenever an admissible heuristic can be defined.

REFERENCES

- [1] Kartik, "Dijkstra's algorithm," https://canvas.mdu.se/courses/18688/files/2895045?module_item_id=677798, 2026, accessed 21-05-2026.
- [2] —, "A* search algorithm," <https://www.geeksforgeeks.org/dsa/a-search-algorithm/>, 2026, accessed 21-05-2026.
- [3] MDU - ELA408, "Lab 3 - path planning," https://canvas.mdu.se/courses/18688/assignments/116235?module_item_id=645519, accessed 21-05-2026.
- [4] M. A. Khan, "A comprehensive study of dijkstra's algorithm," Ph.D. dissertation, 09 2023.
- [5] T. Hossain, S. Uddin, and A. Hasan, "Comprehensive review of the a * algorithm: Theory, implementation, and applications," 09 2021.

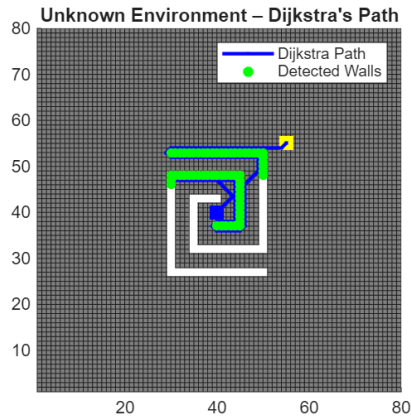


Fig. 3. Dijkstra path planning for an unknown environment.

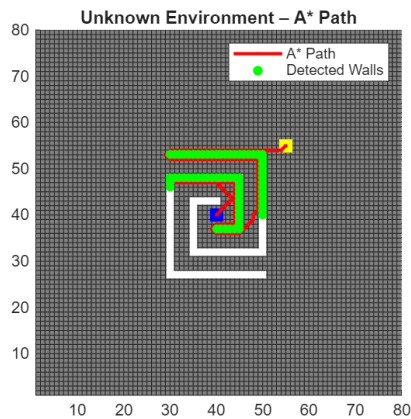


Fig. 4. A* path planning for an unknown environment.

TABLE II
NUMBER OF PUSHES AND POPS - UNKNOWN ENVIRONMENT.

Algorithm	Pushes	Popps
Dijkstra	68489	60905
A*	9285	5754